Predicting AWS Transactions to Mitigate Fraudulent Activity

Presented by Raven Otero-Symphony
Imaginary AWS Data Science Employee

Q1 / March 2026

CNM Ingenuity: Data Science Bootcamp

Important Note: The AWS employment scenario in this presentation is an imaginary application of Project 1, and hopefully a useful tactic to keep the class awake.

Agenda

1 Problem Statement

2 Methodology

3 Key Observations & Assumptions

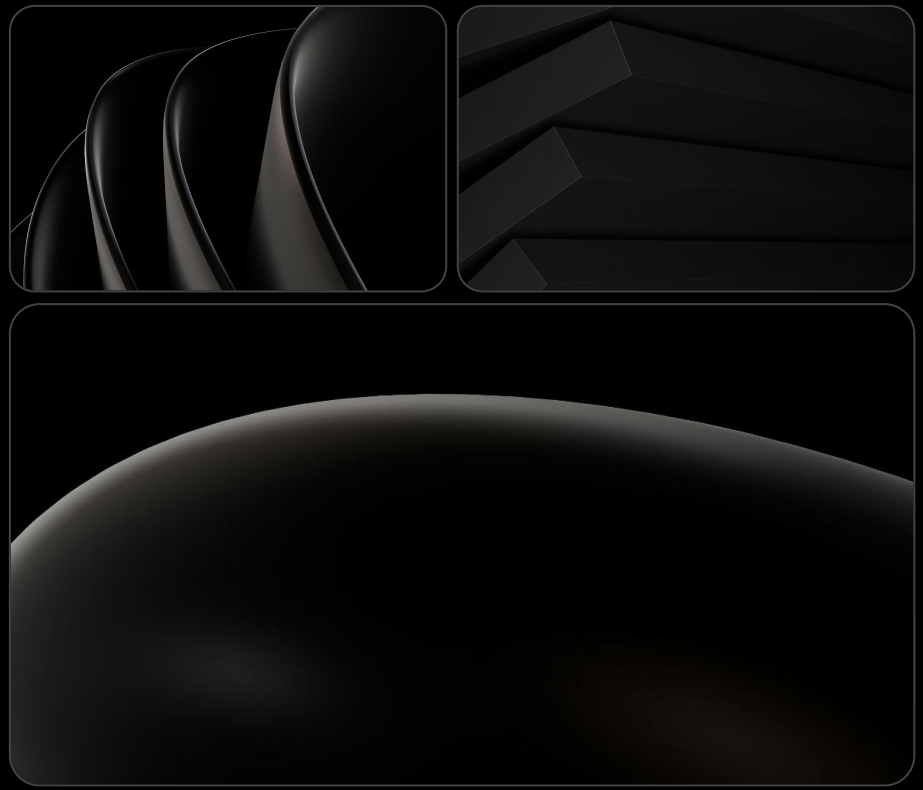
4 GNB Model Predictions

5 Business Implications

6 Questions

Can we *correctly* predict transactions given customer behavior?

- With millions of customers daily, there is growing concern from AWS leadership about fraudulent activity.
- Correctly predicting transaction activity will provide the basis to enhance capabilities for the **Fraud Detection Division**, a new and growing sub-unit of AWS.
- The **Data Science Division** has been tasked with predicting transaction activity and reporting it to the new **Fraud Detection Lead**.



Given transaction data, we apply the `sklearn` Gaussian Naïve Bayes model as a basis to predict transaction activity.

1

Constraint #1:
Binary Classification
Last month's customer transaction data is **continuous** with pre-encoded **binary outputs**, where "1" indicates a transaction and "0" indicates no transaction activity.

2

Constraint #2:
Fewer Training Samples
Given time constraints, we select this model as the **assumption of feature independence** allows agile testing before proceeding with more complex analyses.

3

Constraint #3:
Computational Efficiency
This model estimates probability by assuming **normal distribution of features**, simplifying calculations in Python and Google Colab for our team to iterate on solutions faster.

To ensure accurate model performance, we apply standard data science methods to prepare the data for training.

1

Data Collection

`.head() & .tail()` - What does the original data look like?

`.describe()` - What are the original summary statistics?

`.value_counts()` - What is the count and frequency of our target variable?

`.isna()` - How many features are missing data, and what should we do about it?

2

Data Cleaning

`.copy()` - Create a working copy of the data.

`.dropna()` - Drop index, primary key, and empty features.

`.isna()` - Ensure the target variable does not contain missing values (note: we can't predict on values that don't have a known value).

3

Exploratory Data Analysis

`plotly` - Visualize the distribution of the target and feature variables.

`.corr()` - Investigate correlation between features to validate model approach.

4

Imbalanced Prediction

`sklearn` - Train and test the data using an 80/20 split.

`results.mean()` - Iterate 100 times and return the mean model performance.

`ConfusionMatrixDisplay()` - Visualize true/false positive/negative outputs from the model.

5

Balanced Prediction

The Balanced Prediction follows the same steps as (4), however we coerce the data by resampling it into equal output classes.

Due to the high amount of missing data, we opt for removal rather than imputation.

83%

Missing values in `target` (original)

102

Potentially Usable Features (original)

1.05M rows
104 columns

Dataset Size (original)

	count	mean	std	min	
Unnamed: 0	1050000.0	524999.500000	303109.035663	0.0000	26
target	180000.0	0.100222	0.300297	0.0000	
var_0	180000.0	7.440715	3.021636	-2.6917	
var_1	180000.0	15.870166	3.010492	6.2993	
var_2	180000.0	12.735619	4.185923	-1.0185	
...	...	...	...	...	
var_96	0.0	NaN	NaN	NaN	
var_97	0.0	NaN	NaN	NaN	
var_98	0.0	NaN	NaN	NaN	
var_99	0.0	NaN	NaN	NaN	
var_100	180000.0	14.224538	0.171058	13.7290	

03 rows × 8 columns

Out of 102 features, 51 are completely empty (n = 1,050,000) and the other 51 are somewhat empty (n = 87,000). The only non-null columns (n=2) are the index and primary key.

With only 10% of positive transactions occurring, we validate model assumptions before deploying.

90%

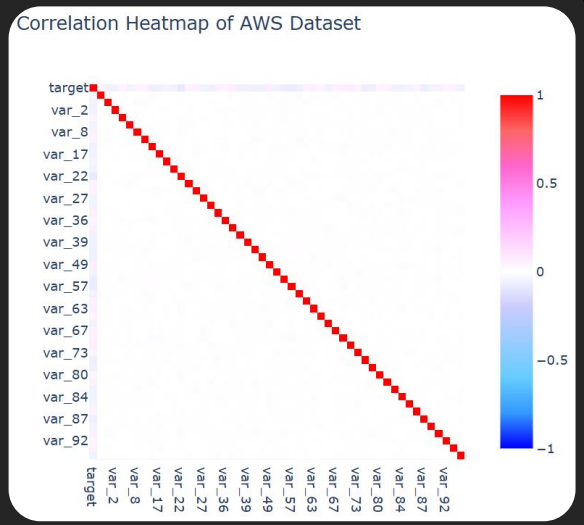
Non-Transaction ("0") `target` data

50

Usable Non-Null Features

180k rows
51 columns

Recovered Dataset Size

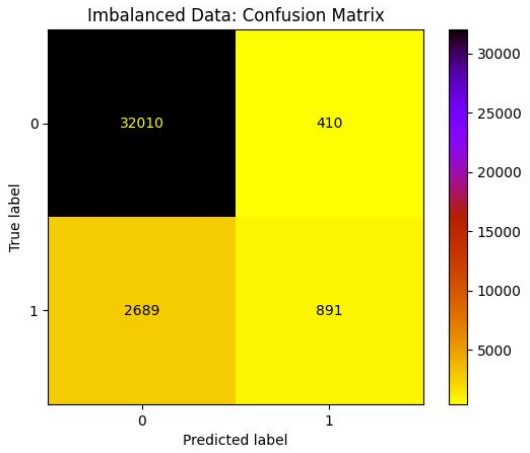


The correlation heatmap of the recovered data (above) doesn't produce feature correlations greater than 5%. However, the target variable is imbalanced, which will skew model output.

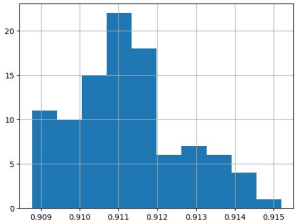
For a subset of customers, our model accurately classifies behavior 76% of the time.

Imbalanced

Mean Performance: 91%

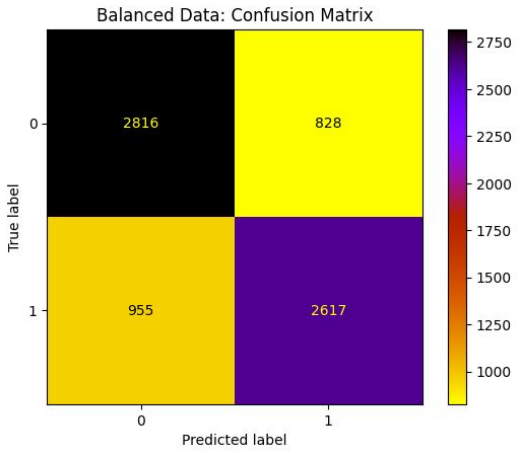


Accuracy: 86%
Precision: 99%
Recall: 92%
F-1 Score: 95%

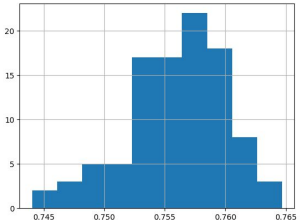


Balanced

Mean Performance: 76%

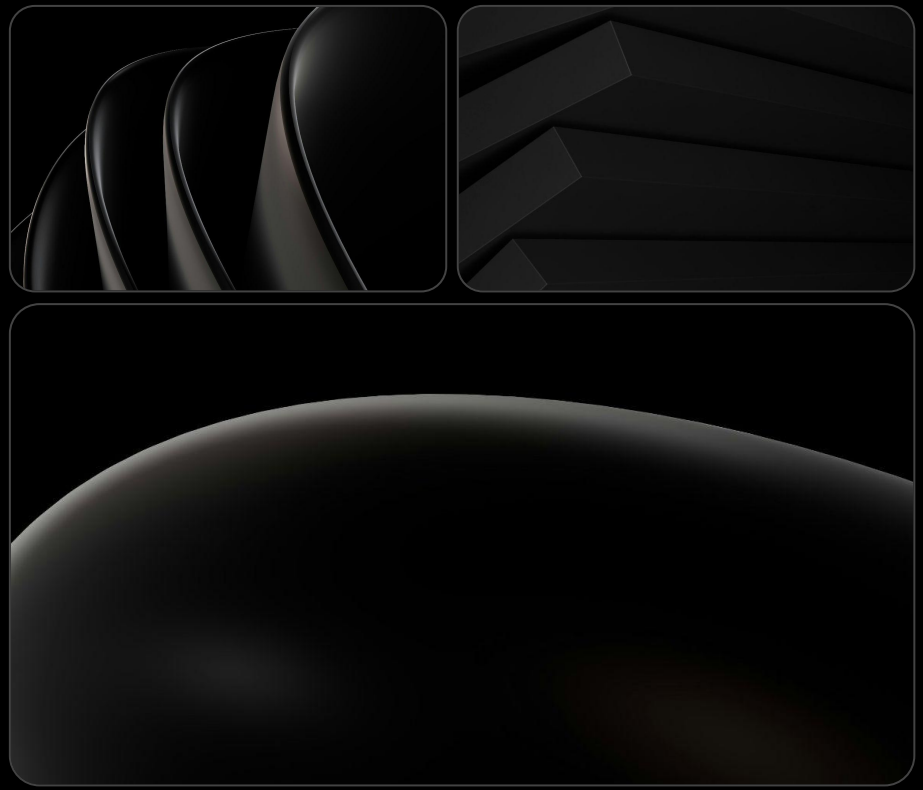


Accuracy: 74%
Precision: 77%
Recall: 75%
F-1 Score: 76%



Further development is needed before going operational.

- To improve data accuracy, more information about the data is required such as fuller customer profiles, documentation of features, timestamps, etc.
- In addition to further Gaussian Naive Bayes model iterations (such as a 60/40 split or feature transformation), additional methods can now be explored including XGBoost or Random Forest.
- The **Product Team** may want to investigate why so many customers stop before making a transaction.



Thank you!

View Codebase Here

Feedback Appreciated

Let's connect!

- GitHub
- LinkedIn